PROGRAMAÇÃO AVANÇADA · UFRN · 2026.2

Árvore Geradora Mínima

Algoritmos de Kruskal e Prim, com exemplo em Python

Este material explica como **conectar todos os vértices de um grafo pelo menor custo** e mostra duas soluções clássicas, **Kruskal** e **Prim**, com um programa em Python (`kruskal_prim.py`) que as executa passo a passo. O exemplo é independente de qualquer outro projeto.

1. O modelo: um grafo é uma rede de relações

Um grafo é uma estrutura matemática que representa objetos e as conexões entre eles. O contexto muda, mas a lógica de pontos e conexões permanece.

Termo	Significado	Exemplo
Vértice (nó)	Os objetos da rede	cidades, computadores, prédios
Aresta	A ligação entre dois vértices	estradas, cabos
Peso	O valor de cada aresta	distância, tempo, custo

2. O problema: conectar tudo pelo menor custo

Dado um grafo **conexo, não direcionado e com pesos**, queremos escolher arestas que:

1. **conectem todos os vértices**;
2. tenham a **menor soma de pesos possível**;
3. **não formem ciclos** (loops).

> **RESULTADO**
> A **Árvore Geradora Mínima (AGM)**, em inglês *Minimum Spanning Tree* (MST): o conjunto de arestas que atende às três condições acima.

- Uma árvore geradora de um grafo com **V** vértices tem exatamente **V - 1** arestas.
- Se todos os pesos forem diferentes, a AGM é **única**. Com pesos repetidos pode haver mais de uma, mas todas têm o mesmo custo.
- Se o grafo for **desconexo**, não existe árvore geradora (só uma floresta).
- Os dois algoritmos são **gulosos**: a cada passo escolhem a aresta mais barata que não estraga a solução. Isso funciona pela *propriedade do corte*: em qualquer divisão dos vértices em dois grupos, a aresta mais leve que cruza a divisão pertence a alguma AGM.

3. Kruskal: gulosa e global

O Kruskal olha para **todas as arestas** do grafo. Começa com uma floresta de vértices isolados, que vai se unindo aos poucos.

1. **Ordenar** todas as arestas, da mais barata à mais cara.
2. **Adicionar** as arestas uma a uma à árvore.
3. **Evitar ciclos:** ignorar a aresta que liga dois vértices que já estão no mesmo componente (verificado com Union-Find).

O **Union-Find** (conjuntos disjuntos) responde rápido se dois vértices já estão conectados: `find(v)` devolve o componente de v e `union(a, b)` junta dois componentes.

PSEUDOCÓDIGO

```
KRUSKAL(V, E):
    A ← ∅
    para cada v em V: criar conjunto {v}
    ordenar E por peso crescente
    para cada (u, v) em E, nessa ordem:
        se find(u) ≠ find(v):         // componentes diferentes: não fecha ciclo
            A ← A ∪ {(u, v)}
            union(u, v)
    retornar A                        // pode parar ao chegar em V - 1 arestas
```

4. Prim: crescimento local contínuo

O Prim faz **uma única árvore** crescer a partir de um vértice inicial.

1. **Começar** por um vértice inicial qualquer.
2. **Examinar a fronteira:** as arestas que ligam o grupo atual a vértices de fora.
3. **Expandir:** adicionar a de menor peso que alcance um vértice novo e repetir até cobrir todos.

Na implementação, a fronteira fica em uma **fila de prioridade** (heap), que entrega sempre a aresta mais leve.

PSEUDOCÓDIGO

```
PRIM(V, E, início):
    T ← {início};  A ← ∅
    enquanto T ≠ V:
        escolher a aresta (u, v) de menor peso com u ∈ T e v ∉ T
        A ← A ∪ {(u, v)}
        T ← T ∪ {v}
    retornar A
```

5. Kruskal × Prim

Critério	Kruskal	Prim
Estratégia	Gulosa, global	Gulosa, local
Foco	**Arestas**	**Vértices**
Crescimento	Une componentes dispersos (floresta → árvore)	Expande uma única árvore
Estrutura de apoio	Ordenação + Union-Find	Fila de prioridade (heap)
Complexidade	O(E log E)	O(E log V) com heap; O(V^2) com matriz de adjacência
Melhor uso (regra prática)	Grafos **esparsos**	Grafos **densos**

REGRA PRÁTICA, NÃO GARANTIA DE DESEMPENHO

Grafo esparso (poucas arestas) favorece o Kruskal; grafo denso (muitas arestas, perto de V^2) favorece o Prim. A preferência também depende da representação do grafo e da implementação. Os dois sempre produzem uma AGM com o mesmo custo.

6. Exemplo: rede de fibra óptica de um campus

Queremos ligar 7 prédios com fibra óptica, gastando o mínimo. Cada aresta é um trajeto possível e o peso é o custo do cabo (valores **fictícios**, em R$ mil). **CT** = Centro de Tecnologia; **RU** = Restaurante Universitário. Ligar todos os trajetos custaria 100; a AGM custa **43**.

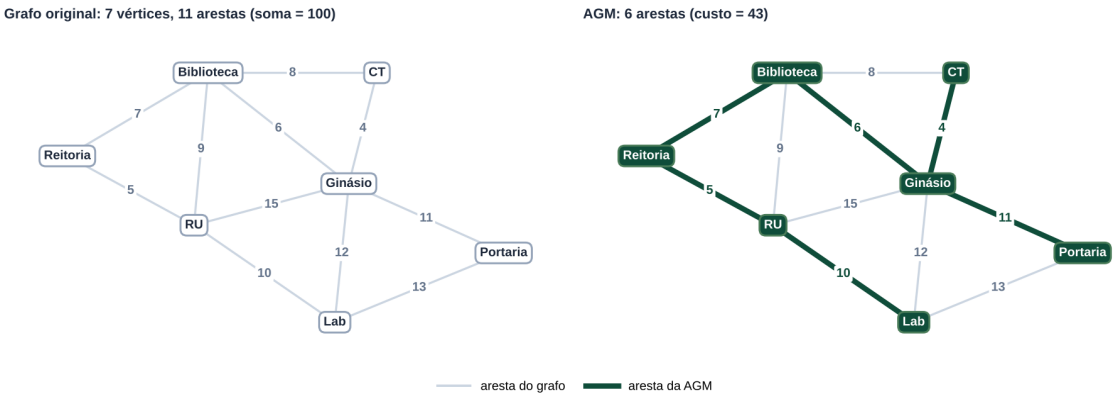


Figura 1 – O grafo do exemplo e a sua AGM (arestas em verde).

6.1 Kruskal passo a passo

O algoritmo para quando a árvore chega a V - 1 = 6 arestas; por isso as arestas de peso 12, 13 e 15 nem chegam a ser examinadas.

#	Aresta	Peso	Decisão	Motivo
1	CT — Ginásio	4	aceita	une dois vértices isolados
2	Reitoria — RU	5	aceita	une dois vértices isolados
3	Biblioteca — Ginásio	6	aceita	Biblioteca entra no componente {CT, Ginásio}
4	Reitoria — Biblioteca	7	aceita	junta {Reitoria, RU} com {Biblioteca, CT, Ginásio}
5	Biblioteca — CT	8	**rejeitada**	mesmo componente (formaria ciclo)
6	Biblioteca — RU	9	**rejeitada**	mesmo componente (formaria ciclo)
7	RU — Lab	10	aceita	Lab entra na árvore
8	Ginásio — Portaria	11	aceita	Portaria entra; 6 arestas, fim

6.2 Prim passo a passo (começando por Reitoria)

#	Fronteira (arestas que saem da árvore)	Escolhida	Peso
1	Reitoria—RU 5, Reitoria—Biblioteca 7	**Reitoria — RU**	5
2	Reitoria—Biblioteca 7, RU—Biblioteca 9, RU—Lab 10, RU—Ginásio 15	**Reitoria — Biblioteca**	7
3	Biblioteca—Ginásio 6, Biblioteca—CT 8, RU—Lab 10, RU—Ginásio 15	**Biblioteca — Ginásio**	6
4	Ginásio—CT 4, Biblioteca—CT 8, RU—Lab 10, Ginásio—Portaria 11, Ginásio—Lab 12	**Ginásio — CT**	4
5	RU—Lab 10, Ginásio—Portaria 11, Ginásio—Lab 12	**RU — Lab**	10
6	Ginásio—Portaria 11, Lab—Portaria 13	**Ginásio — Portaria**	11

6.3 Os passos em imagens

Cada painel mostra o estado depois de uma decisão; as figuras 2 e 3 correspondem às tabelas 6.1 e 6.2. No Prim, algumas arestas ficam "velhas" no heap: o vértice de destino já entrou na árvore por outro caminho. Elas são descartadas ao sair do heap (painéis 5 e 6 da Figura 3).

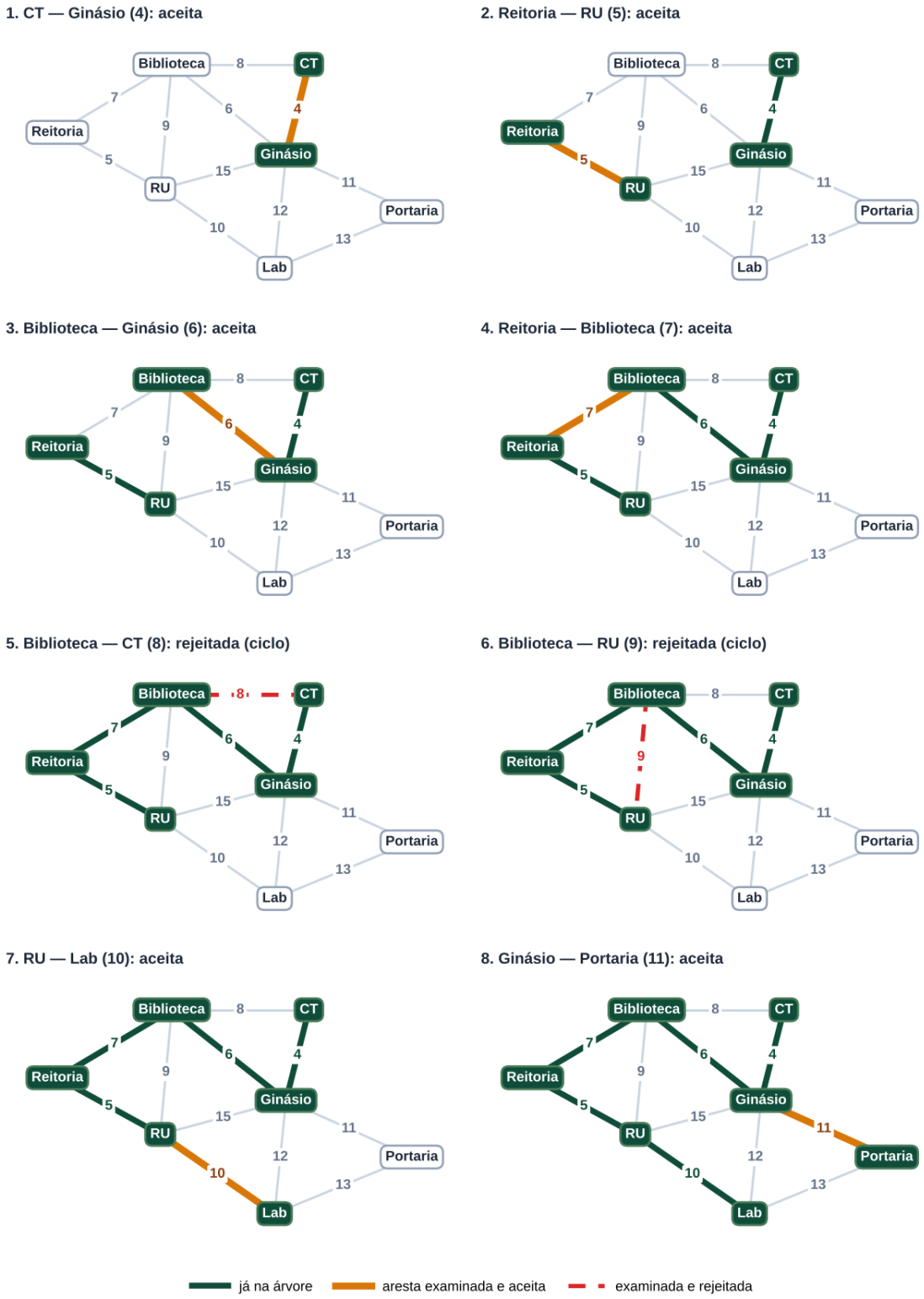


Figura 2 – Kruskal: as 8 arestas examinadas, em ordem crescente de peso.

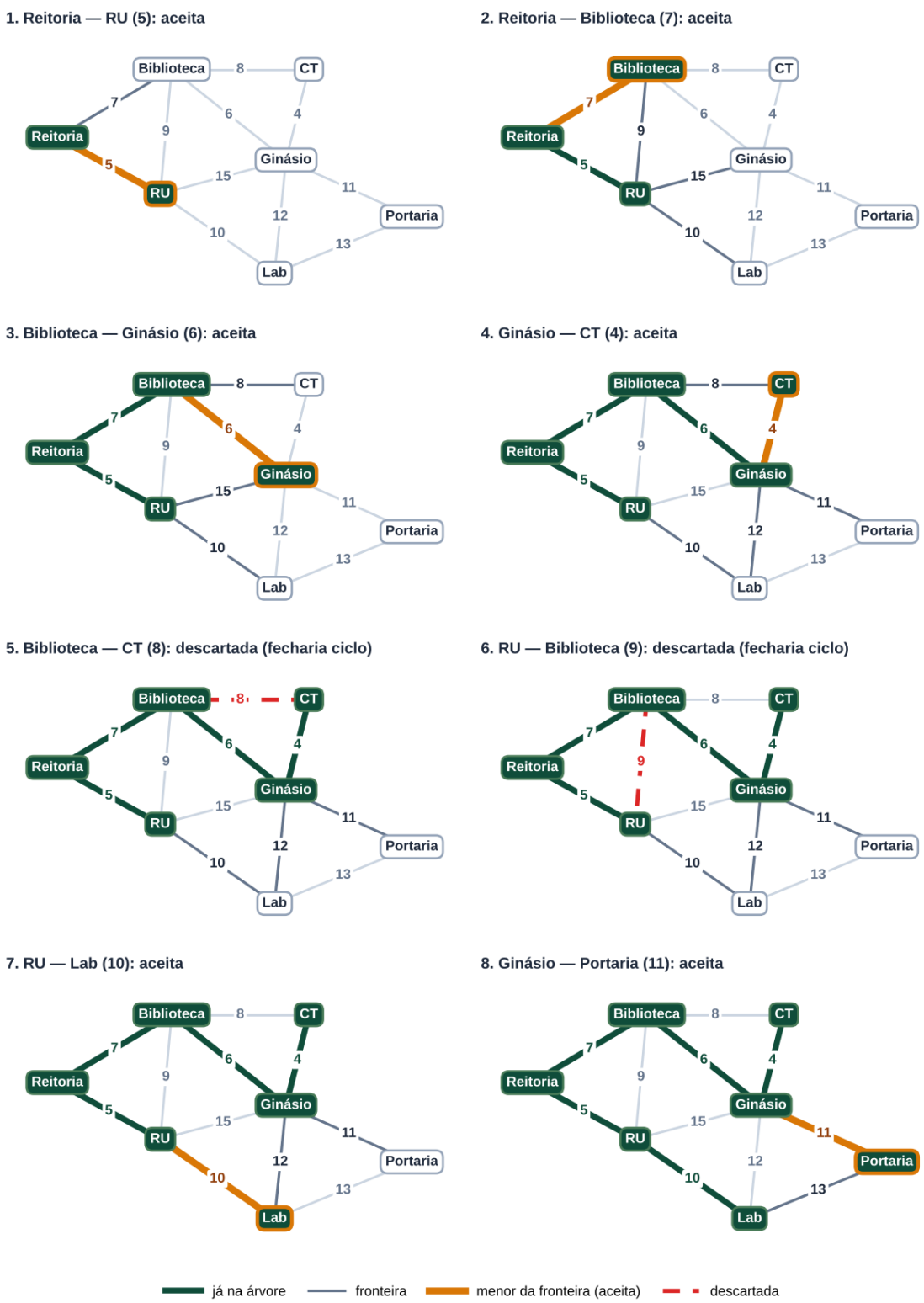


Figura 3 – Prim: a árvore cresce a partir de Reitoria; descartes em vermelho.

6.4 Resultado

Os dois algoritmos escolhem as mesmas 6 arestas, em ordens diferentes, com custo total **43**. Como todos os pesos do exemplo são diferentes, a AGM é única.

Aresta da AGM	Peso (R$ mil)
CT — Ginásio	4
Reitoria — RU	5
Biblioteca — Ginásio	6
Reitoria — Biblioteca	7
RU — Lab	10
Ginásio — Portaria	11
Custo total	**43**

7. Aplicações

- **Infraestrutura:** fibra óptica, tubulações de água e redes elétricas com custo mínimo.
- **Redes de computadores:** o *Spanning Tree Protocol* (STP) evita loops em switches.
- **Machine Learning:** *clustering* baseado em distância.

> **ATENÇÃO**
> O STP constrói uma árvore sem loops, mas **não equivale** a calcular uma AGM por Kruskal ou Prim.

8. Código completo

Arquivo `kruskal_prim.py` (Python 3.9+, somente biblioteca padrão). Para executar: `python kruskal_prim.py`.

Cabeçalho, importações e tipo de aresta

```python
#!/usr/bin/env python3
"""
Árvore Geradora Mínima (AGM) com os algoritmos de Kruskal e Prim.

Exemplo didático: ligar os prédios de um campus com cabo de fibra óptica,
gastando o menor custo total e sem criar ciclos. Os nomes dos prédios e os
custos (em R$ mil) são fictícios e servem apenas para ilustrar.

Uso:
    python kruskal_prim.py

Requisitos: Python 3.9+ (somente biblioteca padrão).
"""
from __future__ import annotations

import heapq

# Uma aresta é a tupla (vértice_u, vértice_v, peso). O grafo é não direcionado.
Aresta = tuple[str, str, int]
```

Union-Find (conjuntos disjuntos), usado pelo Kruskal para detectar ciclos

```python
# ---------------------------------------------------------------------------
# Union-Find (conjuntos disjuntos), usado pelo Kruskal para detectar ciclos
# ---------------------------------------------------------------------------
class UnionFind:
    """Guarda em qual componente cada vértice está.

    find(v)  -> representante (raiz) do componente de v
    union(a, b) -> junta os componentes de a e b; False se já eram o mesmo
    """

    def __init__(self, vertices: list[str]) -> None:
        self.pai = {v: v for v in vertices}      # cada vértice começa sozinho
        self.altura = {v: 0 for v in vertices}   # altura aproximada de cada árvore

    def find(self, v: str) -> str:
        if self.pai[v] != v:
            self.pai[v] = self.find(self.pai[v])  # compressão de caminho
        return self.pai[v]

    def union(self, a: str, b: str) -> bool:
        raiz_a, raiz_b = self.find(a), self.find(b)
        if raiz_a == raiz_b:
            return False                          # já conectados: a aresta fecharia um ciclo
        if self.altura[raiz_a] < self.altura[raiz_b]:
            raiz_a, raiz_b = raiz_b, raiz_a       # a árvore menor entra na maior
        self.pai[raiz_b] = raiz_a
        if self.altura[raiz_a] == self.altura[raiz_b]:
            self.altura[raiz_a] += 1
        return True
```

Kruskal: ordena as arestas e vai juntando componentes

```python
# ---------------------------------------------------------------------------
# Kruskal: ordena as arestas e vai juntando componentes
# ---------------------------------------------------------------------------
def kruskal(vertices: list[str], arestas: list[Aresta],
            verbose: bool = False) -> tuple[list[Aresta], int]:
    """Retorna (arestas da AGM, custo total). Levanta ValueError se o grafo for desconexo."""
    uf = UnionFind(vertices)
    arvore: list[Aresta] = []
    custo = 0

    # 1. ordenar as arestas do menor ao maior peso
    for u, v, peso in sorted(arestas, key=lambda a: a[2]):
        if len(arvore) == len(vertices) - 1:      # árvore completa: V - 1 arestas
            break
        if uf.union(u, v):                        # 2. aceitar se liga componentes distintos
            arvore.append((u, v, peso))
            custo += peso
            if verbose:
                print(f"  + {u} — {v} ({peso}): aceita")
        elif verbose:                             # 3. ignorar se fecha ciclo
            print(f"  x {u} — {v} ({peso}): rejeitada (fecharia um ciclo)")

    if len(arvore) != len(vertices) - 1:
        raise ValueError("grafo desconexo: não existe árvore geradora")
    return arvore, custo
```

Prim: faz uma única árvore crescer a partir de um vértice inicial

```python
# ---------------------------------------------------------------------------
# Prim: faz uma única árvore crescer a partir de um vértice inicial
# ---------------------------------------------------------------------------
def lista_de_adjacencia(vertices: list[str],
                        arestas: list[Aresta]) -> dict[str, list[tuple[str, int]]]:
    adj: dict[str, list[tuple[str, int]]] = {v: [] for v in vertices}
    for u, v, peso in arestas:
        adj[u].append((v, peso))
        adj[v].append((u, peso))
    return adj


def prim(vertices: list[str], arestas: list[Aresta], inicio: str | None = None,
         verbose: bool = False) -> tuple[list[Aresta], int]:
    """Retorna (arestas da AGM, custo total). Levanta ValueError se o grafo for desconexo."""
    adj = lista_de_adjacencia(vertices, arestas)
    # 1. começar por um vértice qualquer
    inicio = inicio if inicio is not None else vertices[0]

    na_arvore = [inicio]
    visitados = {inicio}
    # fronteira: heap com as arestas que saem da árvore, ordenado por peso
    fronteira = [(peso, inicio, v) for v, peso in adj[inicio]]
    heapq.heapify(fronteira)
    arvore: list[Aresta] = []
    custo = 0

    while fronteira and len(visitados) < len(vertices):
        peso, u, v = heapq.heappop(fronteira)    # 2. menor aresta da fronteira
        if v in visitados:                       # aresta "velha": v já entrou na árvore
            if verbose:
                print(f"  - {u} – {v} ({peso}): descartada ({v} já está na árvore)")
            continue
        visitados.add(v)                         # 3. expandir: v entra na árvore
        na_arvore.append(v)
        arvore.append((u, v, peso))
        custo += peso
        if verbose:
            print(f"  + {u} – {v} ({peso}): aceita  | árvore: {', '.join(na_arvore)}")
        for w, p in adj[v]:                                  # novas arestas na fronteira
            if w not in visitados:
                heapq.heappush(fronteira, (p, v, w))

    if len(visitados) != len(vertices):
        raise ValueError("grafo desconexo: não existe árvore geradora")
    return arvore, custo
```

Exemplo: rede de fibra óptica do campus (custos fictícios, em R$ mil)

```python
# ---------------------------------------------------------------------------
# Exemplo: rede de fibra óptica do campus (custos fictícios, em R$ mil)
# CT = Centro de Tecnologia, RU = Restaurante Universitário
# ---------------------------------------------------------------------------
VERTICES = ["Reitoria", "Biblioteca", "CT", "RU", "Ginásio", "Lab", "Portaria"]

ARESTAS: list[Aresta] = [
    ("Reitoria", "Biblioteca", 7),
    ("Reitoria", "RU", 5),
    ("Biblioteca", "CT", 8),
    ("Biblioteca", "RU", 9),
    ("Biblioteca", "Ginásio", 6),
    ("CT", "Ginásio", 4),
    ("RU", "Ginásio", 15),
    ("RU", "Lab", 10),
    ("Ginásio", "Lab", 12),
    ("Ginásio", "Portaria", 11),
    ("Lab", "Portaria", 13),
]
```

```python


def mostrar(titulo: str, arvore: list[Aresta], custo: int) -> None:
    print(f"\n{titulo}")
    for u, v, peso in arvore:
        print(f"  {u} — {v}: {peso}")
    print(f"  Custo total: {custo}")


def main() -> None:
    soma_todas = sum(p for _, _, p in ARESTAS)
    print(f"Grafo: {len(VERTICES)} vértices, {len(ARESTAS)} arestas "
          f"(custo de ligar tudo: {soma_todas}).")

    print("\n=== Kruskal (arestas em ordem crescente de peso) ===")
    arvore_k, custo_k = kruskal(VERTICES, ARESTAS, verbose=True)

    print("\n=== Prim (começando por Reitoria) ===")
    arvore_p, custo_p = prim(VERTICES, ARESTAS, inicio="Reitoria", verbose=True)

    mostrar("AGM por Kruskal:", arvore_k, custo_k)
    mostrar("AGM por Prim:", arvore_p, custo_p)

    # Todo grafo conexo com V vértices tem AGM com exatamente V - 1 arestas.
    assert len(arvore_k) == len(arvore_p) == len(VERTICES) - 1
    # Os dois algoritmos sempre chegam ao mesmo custo mínimo ...
    assert custo_k == custo_p
    # ... e, como os pesos deste exemplo são todos diferentes, a AGM é única:
    # as arestas escolhidas também coincidem (apenas a ordem muda).
    assert {frozenset((u, v)) for u, v, _ in arvore_k} == \
           {frozenset((u, v)) for u, v, _ in arvore_p}
    print(f"\nOK: mesma árvore e mesmo custo ({custo_k}); "
          f"economia de {soma_todas - custo_k} em relação a ligar tudo.")


if __name__ == "__main__":
    main()
```

Saída da execução

```
Grafo: 7 vértices, 11 arestas (custo de ligar tudo: 100).

=== Kruskal (arestas em ordem crescente de peso) ===
  + CT — Ginásio (4): aceita
  + Reitoria — RU (5): aceita
  + Biblioteca — Ginásio (6): aceita
  + Reitoria — Biblioteca (7): aceita
  x Biblioteca — CT (8): rejeitada (fecharia um ciclo)
  x Biblioteca — RU (9): rejeitada (fecharia um ciclo)
  + RU — Lab (10): aceita
  + Ginásio — Portaria (11): aceita

=== Prim (começando por Reitoria) ===
  + Reitoria — RU (5): aceita  | árvore: Reitoria, RU
  + Reitoria — Biblioteca (7): aceita  | árvore: Reitoria, RU, Biblioteca
  + Biblioteca — Ginásio (6): aceita  | árvore: Reitoria, RU, Biblioteca, Ginásio
  + Ginásio — CT (4): aceita  | árvore: Reitoria, RU, Biblioteca, Ginásio, CT
  - Biblioteca — CT (8): descartada (CT já está na árvore)
  - RU — Biblioteca (9): descartada (Biblioteca já está na árvore)
  + RU — Lab (10): aceita  | árvore: Reitoria, RU, Biblioteca, Ginásio, CT, Lab
  + Ginásio — Portaria (11): aceita  | árvore: Reitoria, RU, Biblioteca, Ginásio, CT, Lab, Portaria

AGM por Kruskal:
  CT — Ginásio: 4
  Reitoria — RU: 5
  Biblioteca — Ginásio: 6
  Reitoria — Biblioteca: 7
  RU — Lab: 10
  Ginásio — Portaria: 11
  Custo total: 43

AGM por Prim:
  Reitoria — RU: 5
  Reitoria — Biblioteca: 7
  Biblioteca — Ginásio: 6
  Ginásio — CT: 4
  RU — Lab: 10
  Ginásio — Portaria: 11
  Custo total: 43

OK: mesma árvore e mesmo custo (43); economia de 57 em relação a ligar tudo.
```

Referências

- Material da disciplina: *Teoria dos Grafos e Árvores Geradoras Mínimas* (slides de Programação Avançada, UFRN).
- CORMEN, T. H. et al. *Introduction to Algorithms* (CLRS). Capítulo 23, *Minimum Spanning Trees*.